Building a Clock

Designing a class

Defining a class requires answering two questions:

1. What does it need to remember
2. What does it need to do?

What does a clock need to remember?

`hours`
`minutes`
`seconds`

→ *these become variables in the class*

What does a clock need to do?

move forward one second
print the time (24-hour or 12-hour)

→ *these become member functions*

A class is a type, just like `int`

Defining a class creates a brand-new type. A variable of that type is an **object**.

```
int x = 137;
```

type: `int` variable: `x`

```
Clock myClock;
```

type: `Clock` object: `myClock`

instance variables

The variables that live inside an object. Every object gets its own copy.

member functions (or methods)

The functions that belong to a class, like `incrementSeconds()` and `print24()`.

The Clock class

Here is the whole class declaration.

Same shape as Karel: the `class` keyword, `public:`, the variables, then the member functions.

The style guide says instance variables go at the top of the class, before the functions.

note: we'll get to why we're including iomanip when we talk about printing output

```
1   #include <iostream>
2   #include <iomanip>
3
4   using namespace std;
5
6   class Clock {
7   public:
8       int hours = 0;
9       int minutes = 0;
10      int seconds = 0;
11
12      void incrementSeconds();
13      void incrementMinutes();
14      void incrementHours();
15      void print24();
16      void print12();
17  };
```

Giving instance variables a starting value

`= 0` gives each instance variable a starting value, so every new Clock starts at midnight.

Without it, a brand-new Clock holds whatever garbage happened to be in that memory.

```
1   #include <iostream>
2   #include <iomanip>
3
4   using namespace std;
5
6   class Clock {
7   public:
8       int hours = 0;
9       int minutes = 0;
10      int seconds = 0;
11
12      void incrementSeconds();
13      void incrementMinutes();
14      void incrementHours();
15      void print24();
16      void print12();
17  };
```

Counting up: `incrementSeconds()`

Most of the time, adding a second just means `seconds++`.

But a clock never shows 60 seconds. At 59 it *rolls over*: seconds go back to 0, and the minute goes up by one.

```
19  void Clock::incrementSeconds() {
20      if (seconds == 59) {
21          seconds = 0;
22          incrementMinutes();
23      } else {
24          seconds++;
25      }
26  }
```

Calling member functions from other member functions

Line 22 calls `incrementMinutes()` with no object and no dot in front of it. So which clock's minutes go up?

```
myClock.incrementSeconds();
```



runs `incrementSeconds()` on `myClock`

inside incrementSeconds(), this time:

`seconds`	means	`myClock.seconds`
`incrementMinutes()`	means	`myClock.incrementMinutes()`

A member function can call another member function just by its name. That call runs on the *same* object: `myClock` again.

Rollovers

`incrementMinutes()` rolls over at 59 and bumps the hour.

`incrementHours()` rolls over at 23. It's a 24-hour clock, so after 23:59:59 comes 00:00:00.

So one call to `incrementSeconds()` can set off a chain: seconds, then minutes, then hours.

```
28  void Clock::incrementMinutes() {
29      if (minutes == 59) {
30          minutes = 0;
31          incrementHours();
32      } else {
33          minutes++;
34      }
35  }
36
37  void Clock::incrementHours() {
38      if (hours == 23) {
39          hours = 0;
40      } else {
41          hours++;
42      }
43  }
```

Printing a Clock: `print24()`

First try: print each instance variable with a colon in between.

It works… but that's not what a clock looks like. We want `09:05:03`.

```
45  void Clock::print24() {
46      cout << hours << ":" << minutes << ":" << seconds << endl;
47  }
48
49  int main() {
50      Clock myClock;
51
52      myClock.hours = 9;
53      myClock.minutes = 5;
54      myClock.seconds = 3;
55      myClock.print24();
56
57      return 0;
58  }
```

```
$ ./clock
9:5:3
```

TANGENT: Formatting output

setw and setfill, from <iomanip>

`setw`: set a minimum width

`setw(4)` prints the next value using *at least* 4 characters, padding with spaces on the left.

The columns line up without `\t`.

To use `setw`:

```
#include <iomanip>
```

```
16  int main() {
17      int twoD[ARRAY_SIZE][ARRAY_SIZE];
18      arrayInitializer(twoD);
19
20      for (int row = 0; row < ARRAY_SIZE; row++) {
21          for (int column = 0; column < ARRAY_SIZE; column++) {
22              cout << setw(4) << twoD[row][column];
23          }
24          cout << endl;
25      }
26
27      return 0;
28  }
```

```
$ ./multiply
   0   0   0   0   0
   0   1   2   3   4
   0   2   4   6   8
   0   3   6   9  12
   0   4   8  12  16
```

`setw` only lasts for one item

We need a `setw(2)` in front of *each* number: `setw` applies only to the very next thing printed, then resets.

Better! But the padding is spaces, and clocks don't look like that either.

```
$ ./clock
 9: 5: 3
```

```
45  void Clock::print24() {
46      cout << setw(2) << hours << ":"
47           << setw(2) << minutes << ":"
48           << setw(2) << seconds << endl;
49  }
```

`setfill`: choose the padding character

`setfill('0')` pads with zeros instead of spaces.

Note the single quotes: `setfill` takes a `char`, not a string.

```
$ ./clock
09:05:03
```

```
45  void Clock::print24() {
46      cout << setfill('0');
47      cout << setw(2) << hours << ":"
48           << setw(2) << minutes << ":"
49           << setw(2) << seconds << endl;
50  }
```

Watch out: `setfill` sticks around

After printing the clock, `main()` prints 42 in a width of 6.

We expected 42 with spaces in front. We got `000042`.

```
$ ./clock
09:05:03
000042
```

```
52  int main() {
53      Clock myClock;
54
55      myClock.hours = 9;
56      myClock.minutes = 5;
57      myClock.seconds = 3;
58      myClock.print24();
59
60      cout << setw(6) << 42 << endl;
61
62      return 0;
63  }
```

`setw(n)`

the next item only

`setfill(c)`

everything after, until you change it

Leave cout the way you found it

At the end of `print24()`, set the fill character back to a space.

A good rule for any function: if you change `cout`'s settings, change them back before you return. Whoever calls `print24()` shouldn't get a surprise.

```
$ ./clock
09:05:03
    42
```

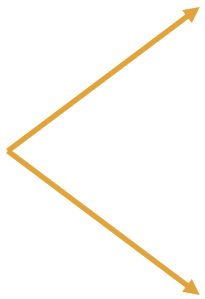
```
45  void Clock::print24() {
46      cout << setfill('0');
47      cout << setw(2) << hours << ":"
48               << setw(2) << minutes << ":"
49               << setw(2) << seconds << endl;
50      cout << setfill(' ');
51  }
```

More than one way to show the time

Most of the world writes `21:05`. The US and a few other countries write `9:05 PM`.
A Clock stores the time *once,* but it can allow that time to be displayed in multiple ways.

inside the Clock

```
hours = 21
minutes = 5
seconds = 3
```



`print24()` **21:05:03**

`print12()` **9:05:03 PM**

First try: `print12()`

`hours % 12` turns 21 into 9.

No `setw` on the hour: 12-hour times are written `9:05 PM`, not `09:05 PM`. Each format has its own padding rules.

Hours before 12 are AM. The rest are PM.

```
53  void Clock::print12() {
54      int displayHour = hours % 12;
55
56      cout << displayHour << ":";
57      cout << setfill('0');
58      cout << setw(2) << minutes << ":"
59           << setw(2) << seconds;
60      cout << setfill(' ');
61
62      if (hours < 12) {
63          cout << " AM" << endl;
64      } else {
65          cout << " PM" << endl;
66      }
67  }
```

Test the edges

Midnight and noon are where clock code breaks, so test them.

Two of these three are wrong. Which ones? What should they say?

```
$ ./clock
9:30:00 PM
0:30:00 AM
0:30:00 PM
```

```
69  int main() {
70      Clock myClock;
71      myClock.minutes = 30;
72
73      myClock.hours = 21;
74      myClock.print12();
75      myClock.hours = 0;
76      myClock.print12();
77      myClock.hours = 12;
78      myClock.print12();
79
80      return 0;
81  }
```

Fixing midnight and noon

`% 12` gives 0 for both midnight and noon. On a 12-hour clock, that hour is written 12.

Testing the edges (0, 12, 23, 59) is how you catch this kind of bug.

```
$ ./clock
9:30:00 PM
12:30:00 AM
12:30:00 PM
```

```
53  void Clock::print12() {
54      int displayHour = hours % 12;
55      if (displayHour == 0) {
56          displayHour = 12;
57      }
58
59      cout << displayHour << ":";
60      cout << setfill('0');
61      cout << setw(2) << minutes << ":"
62              << setw(2) << seconds;
63      cout << setfill(' ');
64
65      if (hours < 12) {
66          cout << " AM" << endl;
67      } else {
68          cout << " PM" << endl;
69      }
70  }
```

Watching it roll over

Start the clock two seconds before midnight, then tick it forward.

One call to `incrementSeconds()` sets off the whole chain: seconds, minutes, *and* hours roll over.

```
$ ./clock
23:59:58
23:59:59
00:00:00
00:00:01
```

```
72  int main() {
73      Clock myClock;
74
75      myClock.hours = 23;
76      myClock.minutes = 59;
77      myClock.seconds = 58;
78
79      for (int tick = 0; tick < 3; tick++) {
80          myClock.print24();
81          myClock.incrementSeconds();
82      }
83      myClock.print24();
84
85      return 0;
86  }
```

Each class has its own instance variables

Like Karel and Lerak: each Clock has its own `hours`, `minutes`, and `seconds`.

We only set `hours`. The others start at 0, thanks to the `= 0` in the class.

Only `workingClock` gets ticked, so only it moves.

```
$ ./clock
09:01:30
09:00:00
```

```
72  int main() {
73      Clock workingClock;
74      workingClock.hours = 9;
75
76      Clock stoppedClock;
77      stoppedClock.hours = 9;
78
79      for (int tick = 0; tick < 90; tick++) {
80          workingClock.incrementSeconds();
81      }
82
83      workingClock.print24();
84      stoppedClock.print24();
85
86      return 0;
87  }
```

Each class has its own instance variables

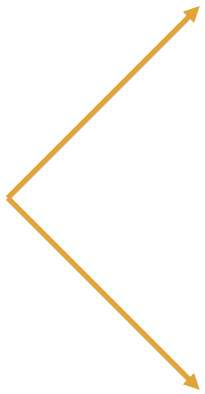
`workingClock` and `stoppedClock` are both **instances** of the `Clock` class.

Each one has its own copy of every **instance variable**. The member functions are shared: they run on whichever object you call them on.

```
class Clock

hours
minutes
seconds

incrementSeconds()
incrementMinutes()
incrementHours()
print24()
print12()
```



workingClock

```
hours = 9    minutes = 1    seconds = 30
```

stoppedClock

```
hours = 9    minutes = 0    seconds = 0
```

the class: a description of a type

Nothing stops users from setting nonsense times

Every instance variable is public, so anyone can set it to anything. It compiles. It runs. It prints `09:75:00`.

And it gets worse: `minutes == 59` will never be true again, so this clock's minutes just keep climbing.

```
$ ./clock
09:75:00
09:76:00
```

```
72  int main() {
73      Clock myClock;
74
75      myClock.hours = 9;
76      myClock.minutes = 75;
77      myClock.print24();
78
79      for (int tick = 0; tick < 60; tick++) {
80          myClock.incrementSeconds();
81      }
82      myClock.print24();
83
84      return 0;
85  }
```

Next time

Making nonsense times impossible